Class Test 1

Regular Expressions and Validation in JavaScript

1. Student Registration Validation: Write a JavaScript program to validate a student registration form containing a full name, email address, 10-digit mobile number, date of birth in DD-MM-YYYY format, and password. The password must contain at least one uppercase letter, one lowercase letter, one digit, and one special character, with a minimum length of eight characters.

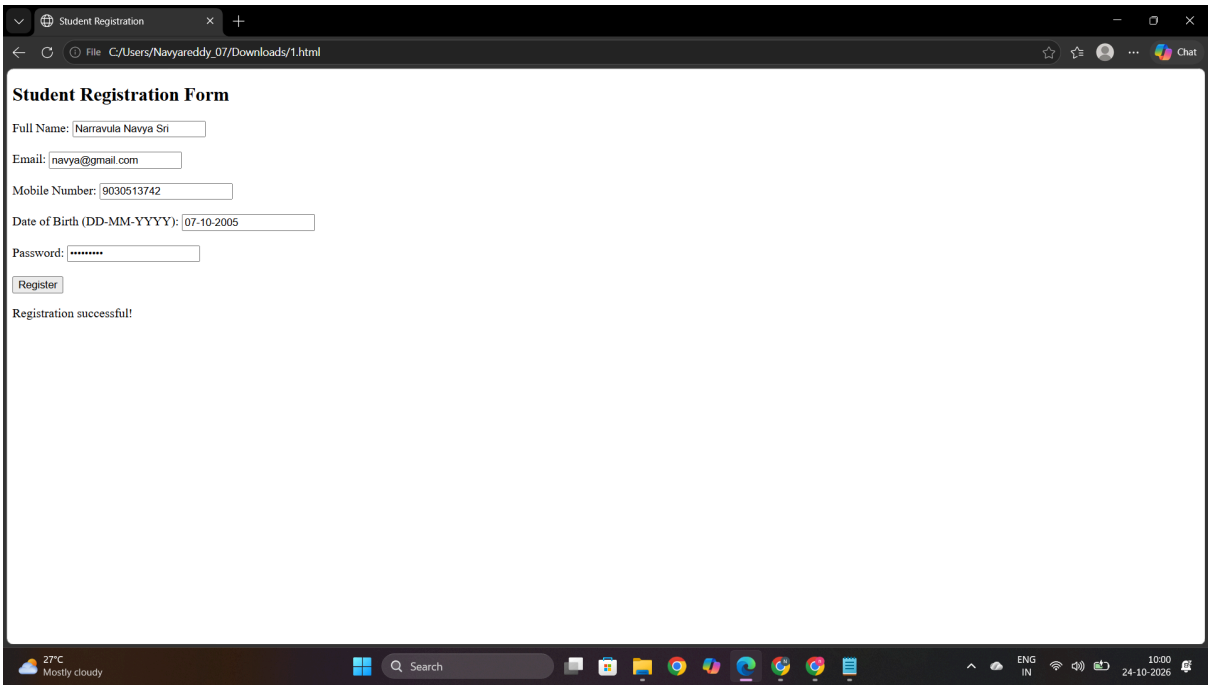
Validates full name, email address, 10-digit mobile number, date of birth in `DD-MM-YYYY` format, and password.

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Student Registration</title>
</head>
<body>
    <h2>Student Registration Form</h2>
    <form id="regForm">
        Full Name:
        <input type="text" id="name" required><br><br>
        Email:
        <input type="text" id="email" required><br><br>
        Mobile Number:
        <input type="text" id="mobile" required><br><br>
        Date of Birth (DD-MM-YYYY):
        <input type="text" id="dob" placeholder="DD-MM-YYYY" required><br><br>
        Password:
        <input type="password" id="password" required><br><br>
        <button type="submit">Register</button>
```

```
</form>
<p id="result"></p>
<script>
    document.getElementById("regForm").addEventListener(
        "submit", function(event) {
        event.preventDefault();
        let name = document.getElementById("name").value.trim();
        let email = document.getElementById("email").value.trim();
        let mobile = document.getElementById("mobile").value.trim();
        let dob = document.getElementById("dob").value.trim();
        let password = document.getElementById("password").value;
        let errors = [];
        if (!/^[A-Za-z]+(?:\s+[A-Za-z]+)+$/.test(name))
            errors.push("Enter a valid full name.");
        if (!/^[^\s@]+@[^\s@.]+(?:\.[^\s@.]+)+$/.test(email))
            errors.push("Enter a valid email address.");
        if (!/^[0-9]{10}$/.test(mobile))
            errors.push("Mobile number must contain 10 digits.");
        let dateMatch = dob.match(
            /^(0[1-9]|[12][0-9]|3[01])-(0[1-9]|1[0-2])-(\d{4})$/
        );
        if (!dateMatch) {
            errors.push("Date must be in DD-MM-YYYY format.");
        } else {
            let day = Number(dateMatch[1]);
            let month = Number(dateMatch[2]);
            let year = Number(dateMatch[3]);
            let date = new Date(year, month - 1, day);
            if (date.getFullYear() !== year ||
                date.getMonth() !== month - 1 ||
                date.getDate() !== day) {
                errors.push("Enter a valid date.");
```

```
            }
        }
        if (password.length < 8 ||
            !/[A-Z]/.test(password) ||
            !/[a-z]/.test(password) ||
            !/[0-9]/.test(password) ||
            !/[^A-Za-z0-9\s]/.test(password)) {
            errors.push(
                "Password needs 8 characters, uppercase, lowercase, digit and special character."
            );
        }
        document.getElementById("result").innerHTML =
            errors.length
            ? errors.join("<br>")
            : "Registration successful!";
    });
  </script>
</body>
</html>
```



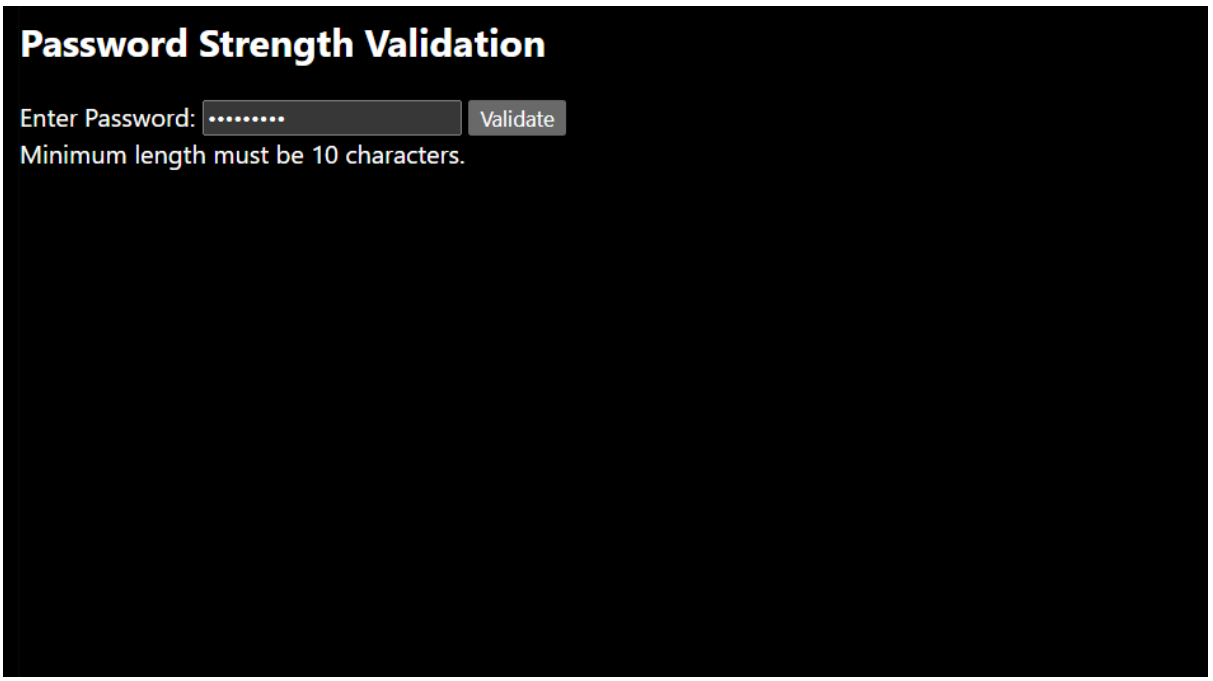
2. Password Strength Validation: Write a JavaScript program to validate a password with the following rules: minimum 10 characters, at least two digits, one uppercase letter, one lowercase letter, one special character, and no whitespace. Display an appropriate error message for each failed condition.

Rules: minimum 10 characters, at least two digits, one uppercase letter, one lowercase letter, one special character, and no whitespace. Each failed condition displays its own error message.

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Password Strength Validation</title>
</head>
<body>
    <h2>Password Strength Validation</h2>
    <label>Enter Password:</label>
    <input type="password" id="pwd">
    <button onclick="validatePassword()">Validate</button>
    <div id="result"></div>
    <script>
        function validatePassword() {
            let password = document.getElementById("pwd").value;
            let errors = [];
            if (password.length < 10)
                errors.push("Minimum length must be 10 characters.");
            if ((password.match(/[0-9]/g) || []).length < 2)
                errors.push("Password must contain at least two digits.");
            if (!/[A-Z]/.test(password))
```

```
            errors.push("Password must contain an uppercase letter.");
        if (!/[a-z]/.test(password))
            errors.push("Password must contain a lowercase letter.");
        if (!/[^A-Za-z0-9\s]/.test(password))
            errors.push("Password must contain a special character.");
        if (/\s/.test(password))
            errors.push("Password must not contain whitespace.");
        document.getElementById("result").innerHTML =
            errors.length
            ? errors.join("<br>")
            : "Password is valid and strong!";
    }
  </script>
</body>
</html>
```



3. Email Address Validation: Write a JavaScript program to validate email addresses according to these rules: the address must contain exactly one @, a non-empty username, a domain name, and a valid-looking extension of at least two letters. Reject spaces and consecutive dots.

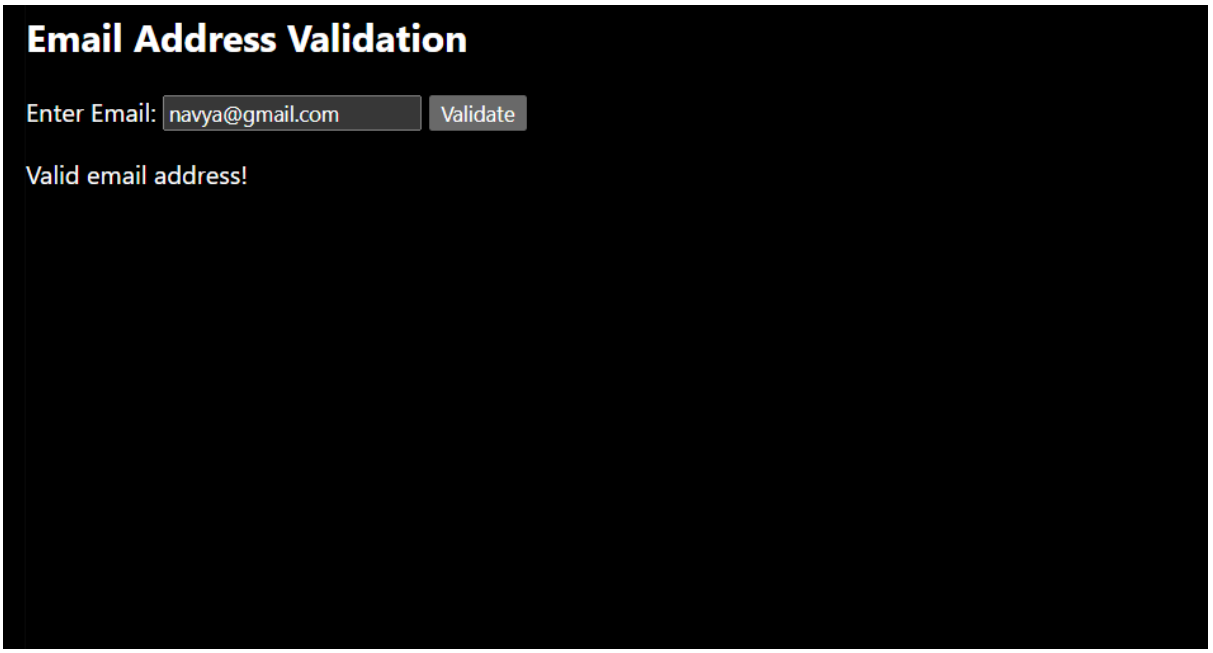
Checks for exactly one @, a non-empty username, a domain name, an extension containing at least two letters, spaces, and consecutive dots.

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Email Validation</title>
</head>
<body>
    <h2>Email Address Validation</h2>
    <label>Enter Email:</label>
    <input type="text" id="email" placeholder="example@gmail.com">
    <button onclick="validateEmail()">Validate</button>
    <p id="result"></p>
    <script>
        function validateEmail() {
            let email = document.getElementById("email").value.trim();
            let pattern =

/^[A-Za-z0-9_%+-]+(?:\.[A-Za-z0-9_%+-]+)*@[A-Za-z0-9](?:[A-Za-z0-9-]*[A-Za-z0-9])?(?:\.[A-Za-z0-9](
?:[A-Za-z0-9-]*[A-Za-z0-9])?)*\.[A-Za-z]{2,}$/;
            let result = document.getElementById("result");
            if (email.includes(" ")) {
                result.textContent = "Email must not contain spaces.";
            } else if ((email.match(/@/g) || []).length !== 1) {
                result.textContent = "Email must contain exactly one @.";
            } else if (email.includes("..")) {
                result.textContent = "Email must not contain consecutive dots.";
            } else if (!pattern.test(email)) {
                result.textContent = "Invalid email address. Check the username, domain and extension.";
```

```
        } else {
            result.textContent = "Valid email address!";
        }
    }
  </script>
</body>
</html>
```



4. Username Validation: Write a JavaScript program to validate a username that must start with a letter, contain only letters, digits, and underscores, have a length of 6–15 characters, and must not end with an underscore or contain consecutive underscores.

Rules: username must start with a letter, contain only letters, digits, and underscores, have 6–15 characters, and must not end with an underscore or contain consecutive underscores.

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

```
    <title>Username Validation</title>
</head>
<body>
    <h2>Username Validation</h2>
    <label>Enter Username:</label>
    <input type="text" id="username" placeholder="Enter username">
    <button onclick="validateUsername()">Validate</button>
    <p id="result"></p>
    <script>
        function validateUsername() {
            let username =
                document.getElementById("username").value;
            let result = document.getElementById("result");
            let pattern = /^[A-Za-z][A-Za-z0-9_]{4,13}[A-Za-z0-9]$/;
            if (username.length < 6 || username.length > 15) {
                result.textContent =
                    "Username must be 6 to 15 characters long.";
            } else if (!/^[A-Za-z]/.test(username)) {
                result.textContent =
                    "Username must start with a letter.";
            } else if (!/^[A-Za-z0-9_]+$/.test(username)) {
                result.textContent =
                    "Use only letters, digits and underscores.";
            } else if (username.endsWith("_")) {
                result.textContent =
                    "Username must not end with an underscore.";
            } else if (username.includes("__")) {
                result.textContent =
                    "Consecutive underscores are not allowed.";
            } else if (!pattern.test(username)) {
                result.textContent = "Invalid username.";
            } else {
```

```
            result.textContent = "Valid username!";
        }
    }
  </script>
</body>
</html>
```



5. Extract and Analyse Data: Write a JavaScript program to extract all mobile numbers, email addresses, and dates from a paragraph using regular expressions. Display each category separately.

This program extracts all mobile numbers, email addresses, and dates from a paragraph and displays each category separately.

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Extract Data Using Regex</title>
</head>
```

```
<body>
    <h2>Extract and Analyse Data</h2>
    <label>Enter Paragraph:</label><br><br>
    <textarea id="paragraph" rows="6" cols="60"
        placeholder="Enter text containing mobile numbers, emails and dates..."></textarea>
    <br><br>
    <button onclick="extractData()">Extract Data</button>
    <h3>Mobile Numbers</h3>
    <ul id="mobiles"></ul>
    <h3>Email Addresses</h3>
    <ul id="emails"></ul>
    <h3>Dates</h3>
    <ul id="dates"></ul>
    <script>
        function extractData() {
            let text = document.getElementById("paragraph").value;
            // Match 10-digit mobile numbers
            let mobilePattern = /(?<!\d)\d{10}(?!\d)/g;
            // Match email addresses
            let emailPattern =
                /[A-Za-z0-9_%+-]+(?:\.[A-Za-z0-9_%+-]+)*@[A-Za-z0-9-]+(?:\.[A-Za-z0-9-]+)*\.[A-Za-z]{2,}/g;
            // Match dates in DD-MM-YYYY format
            let datePattern =
                /\b(?:0[1-9]|[12]\d|3[01])-(?:0[1-9]|1[0-2])-\d{4}\b/g;
            let mobiles = text.match(mobilePattern) || [];
            let emails = text.match(emailPattern) || [];
            let dates = text.match(datePattern) || [];
            displayList("mobiles", mobiles);
            displayList("emails", emails);
            displayList("dates", dates);
        }
        function displayList(id, items) {
```

```
            let list = document.getElementById(id);
            list.replaceChildren();
            if (items.length === 0) {
                let li = document.createElement("li");
                li.textContent = "No data found.";
                list.appendChild(li);
                return;
            }
            items.forEach(function(item) {
                let li = document.createElement("li");
                li.textContent = item;
                list.appendChild(li);
            });
        }
    </script>
</body>
</html>
```

